

Garante Protocol

Cronograma de tareas y guía de bounties · v4

Oct 1, 2026 · Actualizado 6 de oct de 2026 · Alan y Brenda

Todo tiene que estar enviado antes del **13 de octubre de 2026 a las 23:59 (GMT-4)**. Meta interna: enviar a las 18:00 de ese día.

Cómo leer este documento. La sección 1 es el mapa de avance. La 2 es la preparación común: qué instalar, qué cuentas abrir y qué variables de entorno hacen falta. De la 3 a la 9 hay una sección por bounty, **en el orden en que se van a avanzar**, cada una con: qué es, para qué sirve, qué resultado se espera, cómo empezar, el procedimiento técnico y las tareas. La 10 es la construcción del Track 4 y la 11 el calendario.

Qué cambió en la v4

- Reordenado por **bounty**, en el orden de avance, con guía técnica para cada una (antes eran fases sueltas).
- Estado real al 6 de octubre: el front ya tiene registro, cuenta con Mera, firma sin aviso y transacción de prueba, y las pantallas de Tablón, interesados, Mi código y Reputación.
- Preparación común con herramientas, páginas de registro y variables de entorno.
- Sección final de construcción del Track 4.

1. Mapa de avance (6 de oct)

#	Bounty	Premio	Estado	Quién lidera	Siguiente paso
1	Best Mera-Powered UX	\$2,500	En curso	Brenda	Fondos de arranque y sesión por alcance
2	Mera: One Passkey, Many Keys	\$2,500	En curso	Brenda	Derivar las llaves con HKDF
3	Best Use of Envio	\$1,000	Pendiente	Alan	Indexer sobre los eventos de los contratos
4	Best Builds with Qwen 3.8 Max (ERC-8004)	\$5,000 en créditos	Pendiente	Alan	Pedir créditos y registrar el agente
5	Best Workflow with CRE	\$3,000	Pendiente	Alan	Probar <code>cre init</code> y la simulación
6	Best Projects Using Alchemy	\$1,000 en créditos	Pendiente	Brenda	Cambiar el RPC público por el de Alchemy hoy
7	Best Community Team Project	\$5,000	Por confirmar	Brenda	Preguntar en el Discord

Lo que ya está hecho en el front

- Registro con carnet (número y foto) y selfie con cámara: `app/src/app/Registro.jsx`.
- Cuenta con huella (passkey, PRF, cuenta EVM), volver a entrar, firma sin aviso y transacción de prueba: `app/src/lib/mera.js` y `app/src/app/CuentaProvider.jsx`.
- Pantallas de Stitch con datos de ejemplo: `app/src/app/(prestis)/ (/tablon, /tablon/interesados, /mi-codigo, /reputacion)`.
- Commits `d2cabee` y `af3abc2` subidos a main de `BrendaCali/HistorialDePrestamos`.

Lo que falta del lado de Alan (por confirmar entre los dos)

Contratos desplegados (`GaranteIdentity`, `GaranteLedger`, `GaranteReputation`, `GaranteAgentLog`), indexer, SDK, agente y workflow de CRE. Las bounties 3, 4 y 5 dependen de que los contratos estén desplegados y emitan eventos.

2. Preparación común

2.1 Cuentas y páginas donde hay que ingresar

Para qué	Dónde	Qué hacer
MON de prueba (gas)	faucet.monad.xyz	Pedir MON a la dirección de la cuenta o del dispensador
RPC de Alchemy y webhooks	dashboard.alchemy.com	Crear cuenta y una app con red Monad Testnet ; copiar la API key
Qwen 3.8 Max	Alibaba Cloud Model Studio (consola de DashScope)	Crear cuenta, pedir los créditos de la bounty y generar una API key. Las claves son por región: usar la región internacional
Chainlink CRE	docs.chain.link/cre	Crear cuenta de CRE y entrar con <code>cre login</code>
Indexer	envio.dev (Envio Cloud)	Cuenta para desplegar el indexer alojado
Despliegue de la app	vercel.com	Proyecto con el dominio final (la passkey queda atada a él)
Hackathon y ayuda	hackathon.monad.xyz y Discord de Monad Developers	Rol de Metropolis; preguntar por CRE en testnet, Community Team y P256
Docs de Mera	mera.category.xyz y github.com/category-labs/mera	Consulta; guía de passkeys, sesiones y viem
Docs de Monad y ERC-8004	docs.monad.xyz y docs.monad.xyz/guides/erc-8004	Parámetros de red y registros del agente

2.2 Qué instalar

Herramienta	Versión	Para qué	Cómo
Node.js	22 o más (ya tenemos 24)	App Next.js, agente, Envio	nodejs.org
pnpm	última	Envio lo recomienda	<code>npm i -g pnpm</code>
Git y GitHub	—	Commits diarios (los jueces los revisan)	git-scm.com
Docker Desktop	última (en Windows con WSL2)	Correr Envio en local	docker.com
Bun	1.2.21 o más	Workflows de CRE en TypeScript	bun.sh
CRE CLI	última	Crear y simular workflows	docs.chain.link/cre/getting-started/cli-installation
Foundry	1.8 o más	Contratos (Alan)	foundryup
Chrome y un celular	—	Probar passkeys con PRF	Gestor de contraseñas de Google, 1Password o Llavero de iCloud

2.3 Dependencias del proyecto (`package.json` de `app`)

Las librerías se declaran en `app/package.json` con `npm install <paquete>`; las versiones quedan fijadas en `package-lock.json`. Hoy están instaladas:

Paquete	Para qué
<code>next, react, react-dom</code>	La app (Next.js, App Router)
<code>@category-labs/mera</code>	Passkey, PRF, sesiones de firma y adaptador de viem (<code>/viem</code>)

Paquete	Para qué
viem	Cliente de Monad, firma y transacciones
@scure/bip32, @scure/bip39	Derivar la cuenta EVM desde la salida PRF

Se van a sumar: `qrcode` (QR del código dinámico), `openai` (en `agent/`, para llamar a Qwen) y `graphql-request` o simplemente `fetch` (leer el `indexer`).

2.4 Variables de entorno

Van en `app/.env.local` (no se sube al repo). Solo las que empiezan con `NEXT_PUBLIC_` llegan al navegador; **ninguna clave privada lleva ese prefijo**. Hay que agregar los nombres, sin valores, a `.env.example`.

Variable	Quién la usa	Valor / de dónde sale
<code>NEXT_PUBLIC_CHAIN_ID</code>	App	10143 (Monad testnet)
<code>NEXT_PUBLIC_RPC_URL</code>	App, SDK	<code>https://monad-testnet.g.alchemy.com/v2/<KEY></code> (hoy el código usa <code>https://testnet-rpc.monad.xyz</code>)
<code>NEXT_PUBLIC_RP_ID</code>	App	Dominio final de la passkey (en local, localhost)
<code>NEXT_PUBLIC_IDENTITY_ADDRESS,</code> <code>_LEDGER_ADDRESS,</code> <code>_REPUTATION_ADDRESS</code>	App, indexer	Direcciones de los contratos desplegados (las da Alan)
<code>FAUCET_PRIVATE_KEY</code>	Ruta <code>/api/fondos</code> (servidor)	Cuenta de prueba con MON; solo en <code>.env.local</code> y Vercel
<code>ENVIO_GRAPHQL_URL</code>	App, SDK, agente	URL del GraphQL del indexer
<code>DASHSCOPE_API_KEY,</code> <code>DASHSCOPE_BASE_URL,</code> <code>QWEN_MODEL</code>	Agente	Consola de Model Studio. Base URL internacional: <code>https://dashscope-intl.aliyuncs.com/compatible-mode/v1</code>
<code>AGENT_PRIVATE_KEY</code>	Agente	Wallet del agente (la que firma los veredictos)
<code>ALCHEMY_WEBHOOK_SIGNING_KEY</code>	Ruta <code>/api/avisos</code>	Se ve al crear el webhook en el dashboard
<code>CRE_ETH_PRIVATE_KEY</code>	Proyecto de CRE (<code>.env</code> propio)	64 caracteres hex, sin <code>0x</code> ; nunca en el repo

Red de Monad testnet: chain ID 10143, moneda MON, RPC público `https://testnet-rpc.monad.xyz`, explorer `testnet.monadexplorer.com`. Está definida en `app/src/lib/mera.js` (`monadTestnet`).

3. Bounty 1 · Best Mera-Powered UX on Monad

Campo	Detalle
Patrocinador y premio	Monad Foundation · \$2,500 (premio único) · todas las pistas
Qué es	Una app en Monad donde Mera es toda la capa de cuenta : sin frase semilla, sin extensión de billetera y sin servidor de custodia. Gana la app en la que el usuario no nota que hay una blockchain.
Para qué lo hacemos	Es el corazón del producto: toda persona entra, firma y recupera su cuenta solo con su huella. Es la bounty con más probabilidad de ganar porque ya está medio hecha.
Resultado esperado	Registro con una sola ceremonia de passkey hasta la primera transacción confirmada en Monad; sesiones con alcance claro; y la prueba stateless : los jueces borran el almacenamiento o abren otro dispositivo y la identidad se reconstruye solo con la passkey.
Criterios de los jueces	Tiempo hasta la primera transacción (toques y segundos) · diseño de sesión (qué firma sin aviso y qué pide huella) · prueba stateless · bonus por combinar Mera con el resto del stack (patrocinio de gas, recuperación, smart accounts).
Deadline	13 de octubre, 23:59 GMT-4. La selección del panel se puede editar hasta la entrega final.

Cómo empezar

1. Entender el flujo con la guía oficial (mera.category.xyz/getting-started): crear passkey → obtener 32 bytes PRF → derivar cuenta → sesión de firma → `session.end()`. Esto ya está en el código.
2. Requisitos: contexto seguro (HTTPS o `localhost`) y un autenticador con PRF (1Password, Llavero de iCloud o Gestor de contraseñas de Google). En Chrome de escritorio la primera vez el Gestor de Google pide crear un PIN de recuperación.
3. Probar el flujo que ya existe: `cd app && npm run dev`, abrir `http://localhost:3000`, completar carnet y selfie, y usar "Crear cuenta con mi huella".

Procedimiento de implementación

Paso 1 · Cuenta y sesión (hecho)

`app/src/lib/mera.js` exporta `crearCuenta()`, `entrarConHuella()`, `saldoMon()` y `enviarPrueba()`. `abrirSesion()` convierte la salida PRF en la cuenta EVM (ruta `m/44'/60'/0'/0/0`) y devuelve la sesión, la dirección y la cuenta de viem. `CuentaProvider.jsx` guarda la sesión **solo en memoria** (un `useRef`) y expone `crear`, `entrar`, `firmar`, `enviar` y `salir`.

Paso 2 · Fondos de arranque

Una cuenta nueva no tiene MON. Crear `app/src/app/api/fondos/route.js`. Para no usar base de datos, el límite por dirección se resuelve con el saldo: solo se envía si la dirección tiene menos de un umbral.

```
// app/src/app/api/fondos/route.js
import { createWalletClient, createPublicClient, http, parseEther, isAddress } from 'viem';
import { privateKeyToAccount } from 'viem/accounts';
import { monadTestnet } from '../../lib/mera';

export async function POST(req) {
  const { direccion } = await req.json();
  if (!isAddress(direccion)) return Response.json({ error: 'direccion invalida' }, { status: 400 });
  const publico = createPublicClient({ chain: monadTestnet, transport: http(process.env.NEXT_PUBLIC_RPC_URL) });
  if ((await publico.getBalance({ address: direccion })) > parseEther('0.05'))
    return Response.json({ ok: true, enviado: false }); // ya tiene fondos
  const cuenta = privateKeyToAccount(process.env.FAUCET_PRIVATE_KEY);
```

```
const wallet = createWalletClient({ account: cuenta, chain: monadTestnet, transport:
http(process.env.NEXT_PUBLIC_RPC_URL) });
const hash = await wallet.sendTransaction({ to: direccion, value: parseEther('0.1') });
await publico.waitForTransactionReceipt({ hash });
return Response.json({ ok: true, enviado: true, hash });
}
```

Después, en `Registro.jsx`, llamar a esta ruta justo después de `crear()` y antes de la primera transacción. Un solo clic del usuario debe encadenar: crear passkey → fondos → primera transacción.

Paso 3 · Primera transacción con medición

Hoy `enviarPrueba()` manda 0 MON a la propia cuenta. Cuando `GaranteIdentity` esté desplegado, reemplazarla por el registro real (`registrar(hashCarnet)`) con `wallet.writeContract(...)`. Medir con `performance.now()` desde que se pulsa "Crear cuenta" hasta el recibo, y contar los toques; ese número va al texto y al video de la bounty.

Paso 4 · Sesiones por alcance

- **Sin aviso:** ver tablón, perfiles, buscar y "Me interesa" usan la sesión en memoria.
- **Con huella cada vez:** abrir o confirmar un préstamo, calificar y revelar. En `CuentaProvider.jsx` agregar `firmarAccion(tipoDatos)`, que cierra la sesión (`session.end()`), llama a `entrarConHuella()` para una huella fresca, firma el EIP-712 con `cuenta.signTypedData(...)` y vuelve a cerrar.
- **Inactividad:** un temporizador que llame a `salir()` a los pocos minutos y deje un estado `sesionExpirada` que muestre el botón "Continuar con huella".
- **Rutas protegidas:** hoy `/tablon`, `/mi-codigo` y `/reputacion` se abren sin sesión. En `app/src/app/(prestis)/layout.jsx`, si no hay `direccion`, mostrar la pantalla de "Continuar con huella".

Paso 5 · Modo juez

Reemplazar el botón "Soy juez" de `Registro.jsx` (hoy entra directo al Tablón). Nuevo flujo: el juez **crea su propia cuenta** con su huella (sin carnet boliviano, el verificador simulado dice que sí), y una pantalla "Cargar historial de ejemplo" carga 2 o 3 historiales de muestra marcados como tales. Las cuentas compartidas no pasan la prueba stateless, por eso cada juez crea la suya.

Paso 6 · Dominio y prueba stateless

- Fijar el dominio final en Vercel y guardarlo en `NEXT_PUBLIC_RPC_ID`; usar esa variable en `rpId()` de `mera.js`. Una passkey creada en `localhost` no sirve en otro dominio.
- Prueba: crear cuenta, anotar la dirección, borrar el almacenamiento del sitio (DevTools → Application → Clear site data), recargar, "Ya tengo cuenta" y comprobar la misma dirección y el mismo perfil.
- Comprobar que no se guarda nada en el navegador: `grep -rn "localStorage\|sessionStorage\|indexedDB" app/src`. Repetir después de cada despliegue.
- Probar en celular y en Chrome de escritorio, y elegir un autenticador que muestre **un solo aviso** al crear la cuenta.

Tareas

Tarea	Quién	Estado	Dónde
Cuenta con huella, volver a entrar, firma sin aviso	Brenda	Hecho	<code>src/lib/mera.js</code> , <code>CuentaProvider.jsx</code>
Transacción de prueba con tiempo medido	Brenda	Hecho	<code>Registro.jsx</code> (probarTx)
Ruta de fondos de arranque	Brenda	Pendiente	<code>src/app/api/fondos/route.js</code>
Encadenar crear cuenta → fondos → primera transacción	Brenda	Pendiente	<code>Registro.jsx</code>
Sesión por alcance, inactividad y rutas protegidas	Brenda	Pendiente	<code>CuentaProvider.jsx</code> , <code>(prestis)/layout.jsx</code>

Tarea	Quién	Estado	Dónde
firmarAccion con EIP-712 (cuando Alan pase el tipo)	Brenda	Pendiente	src/lib/prestamo.js (nuevo)
Modo juez con cuenta propia y datos de muestra	Brenda	Pendiente	Registro.jsx, src/app/juez/page.jsx (nuevo)
Dominio final y prueba en celular y Chrome	Los dos	Pendiente	Vercel
Prueba stateless en el despliegue final	Los dos	Pendiente	—
Texto de integración y video de 2 min	Los dos	Pendiente	12 de oct

Texto de entrega (borrador)

"Garante usa Mera como toda la capa de cuenta. El registro es una sola ceremonia de passkey: de la salida PRF se deriva la cuenta EVM, la sesión de firma vive solo en memoria y firma sin aviso las acciones de lectura; abrir, confirmar o calificar un préstamo exige una huella fresca. No hay frase semilla, extensión ni servidor de custodia, y la identidad se reconstruye en cualquier dispositivo solo con la passkey."

4. Bounty 2 · Mera: One Passkey, Many Keys

Campo	Detalle
Patrocinador y premio	Monad Foundation · \$2,500 · todas las pistas
Qué es	El uso más creativo, fuera de una billetera , del material de clave que sale del PRF de Mera.
Para qué lo hacemos	De la misma huella sacamos cuatro cosas distintas: la cuenta, una llave que cifra datos privados, la sal de la calificación a ciegas y la clave del código dinámico. Así el usuario no administra nada y todo se reconstruye con la passkey.
Resultado esperado	Una función que, a partir de la salida PRF, devuelve las cuatro llaves; el monto exacto de un préstamo cifrado que solo ven las partes; una calificación a ciegas cuya sal se reconstruye en otro dispositivo; y un QR firmado que cambia cada 60 s.
Cómo se relaciona con la 1	Comparten la misma passkey y la misma ceremonia. Esta bounty solo agrega las llaves derivadas.

Cómo empezar

1. Leer en la documentación de Mera "Llaves y cuentas" (mera.category.xyz/concepts/entropy-keys-and-accounts), que explica la derivación determinista.
2. Entender HKDF: derivar claves independientes de un secreto con una **etiqueta distinta** por uso. Con etiquetas distintas, la llave de cifrado no se puede confundir con la de la sal.
3. No hace falta librería nueva: el navegador trae `crypto.subtle` (HKDF, AES-GCM). La salida PRF no se guarda; vive solo dentro de `abrirSesion()`.

Procedimiento de implementación

Paso 1 · Derivar las llaves

En `app/src/lib/mera.js`, junto a `abrirSesion()`:

```
async function derivar(prfOutput, etiqueta) {
  const base = await crypto.subtle.importKey('raw', prfOutput, 'HKDF', false, ['deriveBits']);
  const bits = await crypto.subtle.deriveBits(
    { name: 'HKDF', hash: 'SHA-256', salt: new Uint8Array(0), info: new
    TextEncoder().encode(etiqueta) },
    base, 256);
  return new Uint8Array(bits);
}

// dentro de abrirSesion(prfOutput):
const llaveCifrado = await derivar(prfOutput, 'garante/cifrado/v1');
const salCommit = await derivar(prfOutput, 'garante/commit/v1');
const llaveCodigo = await derivar(prfOutput, 'garante/codigo/v1');
```

`abrirSesion` pasa a ser `async` y devuelve las cuatro llaves junto con la cuenta. `CuentaProvider.jsx` las guarda en el mismo `useRef` que la sesión (memoria, nunca en el navegador) y las borra en `salir()`.

Paso 2 · Cifrar el monto exacto

El monto exacto no va onchain: se cifra con AES-GCM usando `llaveCifrado` y solo se publica el rango. La contraparte lo recibe cifrado con una clave compartida acordada al abrir el préstamo (decisión de Alan: cómo se intercambia esa clave).

```
const clave = await crypto.subtle.importKey('raw', llaveCifrado, 'AES-GCM', false, ['encrypt',
'decrypt']);
const iv = crypto.getRandomValues(new Uint8Array(12));
const cifrado = await crypto.subtle.encrypt({ name: 'AES-GCM', iv }, clave, new
TextEncoder().encode('1000'));
```

Paso 3 · Calificación a ciegas

El commit es `keccak256(calificacion, salCommit)` (con `encodePacked` de `viem`) y el reveal publica la calificación y la sal. Como la sal sale de la passkey, se reconstruye en otro dispositivo sin guardar nada. Pantalla de commit y revelado simultáneo en `app/src/app/(prestis)/ (nueva)`.

Paso 4 · Código dinámico

Reemplazar el relleno aleatorio de `app/src/app/(prestis)/mi-codigo/CodigoDobleFirma.jsx` (hoy usa `Math.random` y una cuenta regresiva fija) por:

- Un payload { `direccion`, `nonce`, `expiracion` } con expiración de 60 s, firmado.
- Un QR del payload con la librería `qrcode` (`npm install qrcode`).
- Un código corto de 6 dígitos que apunta a ese QR a través de un relay temporal (ruta `src/app/api/relay/route.js`, guarda el QR 60 s). El prestamista **no** verifica el TOTP: verifica la firma del QR, y el contrato comprueba `nonce` y expiración.
- Botones del diseño: "Regenerar código ahora" y "Escanear QR de otra persona" (cámara, como la selfie).

Decisión pendiente: si el QR se firma con una clave HKDF distinta de la cuenta, esa dirección hay que registrarla en `GaranteIdentity`. Más simple y suficiente para el MVP: firmar con la cuenta EVM y usar HKDF solo para el cifrado y la sal. Acordarlo con Alan antes de implementarlo.

Tareas

Tarea	Quién	Estado	Dónde
Función <code>derivar</code> y las tres llaves HKDF	Brenda	Pendiente	<code>src/lib/mera.js</code>
Guardar las llaves solo en memoria y borrarlas al salir	Brenda	Pendiente	<code>CuentaProvider.js</code>
Cifrado AES-GCM del monto exacto	Brenda	Pendiente	<code>src/lib/cifrado.js</code> (nuevo)
Sal del commit y pantalla de calificación a ciegas	Brenda	Pendiente	<code>(prestis)/calificar</code> (nuevo)
QR firmado con <code>nonce</code> y expiración de 60 s	Brenda	Pendiente	<code>CodigoDobleFirma.jsx</code>
Relay del código corto de 6 dígitos	Brenda	Pendiente	<code>src/app/api/relay/route.js</code> (nuevo)
Escáner de QR de la otra persona	Brenda	Pendiente	<code>CodigoDobleFirma.jsx</code>
Verificar <code>nonce</code> y expiración en el contrato	Alan	Pendiente	<code>GaranteLedger</code>

Texto de entrega (borrador)

"De una sola passkey derivamos, con HKDF y etiquetas distintas, la cuenta EVM, una llave que cifra el monto exacto de cada préstamo, la sal de la calificación a ciegas (commit-reveal) y la clave que firma el código dinámico de doble firma. Nada se guarda: todo se reconstruye con la huella en cualquier dispositivo."

5. Bounty 3 · Best Use of Envio

Campo	Detalle
Patrocinador y premio	Envio · \$1,000
Qué es	Un indexer (HyperIndex) que sostiene una función principal de la app.
Para qué lo hacemos	El tablón, los perfiles, el buscador, la lista de interesados y el puntaje salen del indexer, no de leer contratos uno por uno. También le da el grafo de préstamos al agente de Qwen y los datos al SDK.
Resultado esperado	Un indexer corriendo sobre los eventos de los contratos que expone por GraphQL: perfil con puntaje y nivel, buscador, interesados por publicación y el grafo de calificaciones.
Depende de	Contratos desplegados en Monad testnet que emitan eventos (Alan).

Cómo empezar

1. Instalar Node 22+, pnpm y **Docker Desktop** (el indexer se corre en local con Docker; en Windows, con WSL2).
2. Tener los ABIs de los contratos (`contracts/out/...json` de Foundry) y las direcciones desplegadas.
3. Leer `docs.envio.dev` → HyperIndex → Quickstart. Un indexer son tres archivos: `config.yaml`, `schema.graphql` y `src/EventHandlers.ts`.

Procedimiento de implementación

Paso 1 · Crear el indexer

```
mkdir indexer && cd indexer
pnpx envio init
```

En el asistente elegir **Contract Import** → **Local ABI**, indicar la ruta del ABI, los eventos a indexar, la red **Monad testnet (10143)**, el nombre del contrato y su dirección. Si Monad testnet no aparece en la lista, se agrega a mano en `config.yaml` con la URL del RPC. Repetir para cada contrato. Esto genera los tres archivos.

Paso 2 · `config.yaml`

```
name: garante-indexer
networks:
  - id: 10143
    start_block: 0          # poner el bloque de despliegue
    contracts:
      - name: GaranteLedger
        address: ['0x...']
        abi_file_path: ../contracts/out/GaranteLedger.sol/GaranteLedger.json
        handler: src/EventHandlers.ts
        events:
          - event: LoanOpened(uint256 indexed id, address indexed lender, address indexed debtor, uint64 due)
          - event: LoanClosed(uint256 indexed id, uint8 result)
```

Los nombres y parámetros de los eventos los define Alan; este bloque es un ejemplo de forma. Agregar los de GaranteReputation (RatingCommitted, RatingRevealed) y GaranteIdentity (Verified).

Paso 3 · `schema.graphql` y manejadores

```
type Persona { id: ID!  verificada: Boolean!  puntaje: Int!  nivel: String!  prestamos: Int!
aTiempo: Int! }
type Prestamo { id: ID!  prestamista: String!  deudor: String!  vence: Int!  estado: String! }
type Calificacion { id: ID!  prestamo: String!  autor: String!  revelada: Boolean! }
type Veredicto { id: ID!  tipo: String!  cuentas: [String!]!  explicacion: String! }
```

```
// src/EventHandlers.ts
import { GaranteLedger } from 'generated';
GaranteLedger.LoanOpened.handler(async ({ event, context }) => {
  context.Prestamo.set({ id: event.params.id.toString(), prestamista: event.params.lender,
    deudor: event.params.debtor, vence: Number(event.params.due), estado: 'Activo' });
});
```

El **puntaje 0–1000** y el **nivel** se recalculan en los manejadores de `LoanClosed` y `RatingRevealed` con la fórmula del documento de la idea (45% a tiempo, 25% gravedad, 15% personas distintas, 10% montos, 5% antigüedad).

Paso 4 · Correrlo y consultarlo

```
pnpm install
pnpm dev # levanta el indexer y el GraphQL local con Docker
```

La consola muestra la URL local del GraphQL. Probar una consulta de perfil desde ahí y guardar la URL en `ENVIO_GRAPHQL_URL`. Para producción, desplegar en Envío Cloud y cambiar la URL.

Paso 5 · Conectarlo a la app

- Tablón y perfil: reemplazar los datos fijos de `(prestis)/tablon/Publicaciones.jsx`, `(prestis)/tablon/interesados/Interesados.jsx` y `(prestis)/reputacion/page.jsx` por consultas al GraphQL.
- Interesados por publicación: decisión de Alan (evento `onchain InterestExpressed` o base `offchain`). Si es `onchain`, también lo indexa Envío.
- Buscador por nombre, código o ciudad con filtros por puntaje y nivel.

Tareas

Tarea	Quién	Estado	Dónde
envio init por contrato y config.yaml	Alan	Pendiente	indexer/config.yaml
Entidades y manejadores de eventos	Alan	Pendiente	indexer/schema.graphql, src/EventHandlers.ts
Cálculo del puntaje 0–1000 y del nivel	Alan	Pendiente	manejadores
Interesados por publicación	Alan	Pendiente	indexer
Despliegue en Envío Cloud	Alan	Pendiente	envio.dev
Conectar tablón, interesados y perfil al GraphQL	Brenda	Pendiente	(prestis)/ y src/lib/indexer.js (nuevo)
Buscador con filtros	Brenda	Pendiente	(prestis)/buscar (nuevo)

6. Bounty 4 · Best Builds with Qwen 3.8 Max (con ERC-8004)

Campo	Detalle
Patrocinador y premio	Alibaba Cloud · \$5,000 en créditos
Qué es	Que Qwen actúe de verdad en Monad: no un chatbot, sino un agente con identidad onchain que hace un trabajo y publica el resultado.
Para qué lo hacemos	Detectar anillos de calificaciones ($A \rightarrow B \rightarrow C \rightarrow A$), explicarlos y publicar el veredicto firmado. También resume perfiles ("José es muy cumplido: 11 de 12 a tiempo") y modera el tablón. Es la defensa contra el puntaje inflado y cumple la parte de infraestructura de IA del Track 4.
Resultado esperado	Con tres cuentas que se califican en círculo, el agente lo detecta, escribe la explicación y publica el veredicto firmado con su identidad ERC-8004 en GaranteAgentLog. El veredicto aparece en el perfil y en el explorer.
Depende de	Envío (3) para leer el grafo, y de GaranteAgentLog desplegado.

Cómo empezar

1. Crear la cuenta de Alibaba Cloud Model Studio, **pedir los créditos de la bounty** y generar una API key. Las claves son por región: usar la región internacional y su URL.
2. Confirmar en la consola el identificador exacto del modelo (el hackathon lo llama "Qwen 3.8 Max") y guardarlo en `QWEN_MODEL`.
3. Leer docs.monad.xyz/guides/erc-8004 y github.com/erc-8004 (contratos y SDK `agent0`, `sdk.ag0.xyz`).
4. Crear la carpeta `agent/` (Node) con `npm init -y` y `npm install openai viem`.

Procedimiento de implementación

Paso 1 · Llamar a Qwen

La API de Model Studio es compatible con el formato de OpenAI, así que se usa el paquete `openai` cambiando la URL base:

```
import OpenAI from 'openai';
const qwen = new OpenAI({ apiKey: process.env.DASHSCOPE_API_KEY, baseUrl:
process.env.DASHSCOPE_BASE_URL });
const r = await qwen.chat.completions.create({
  model: process.env.QWEN_MODEL,
  messages: [{ role: 'user', content: 'Explica en 2 frases por que estas 3 cuentas parecen un
anillo: ...' }],
});
console.log(r.choices[0].message.content);
```

Paso 2 · Detectar anillos en código, explicar con Qwen

La **detección** es un algoritmo determinista (ciclos cortos en el grafo de calificaciones leído de Envío); no se le deja a un modelo. Qwen se usa para **explicar** el hallazgo y resumir perfiles. Así el veredicto es reproducible.

```
// agent/anillos.js (ciclos A->B->C->A en el grafo de calificaciones)
export function anillos(aristas /* {de, a}[] */, maxLargo = 4) {
  const sig = new Map(); for (const { de, a } of aristas) (sig.get(de) ?? sig.set(de,
[]).get(de)).push(a);
  const hallados = new Set();
  const dfs = (ini, nodo, ruta) => {
    for (const s of sig.get(nodo) ?? []) {
      if (s === ini && ruta.length >= 3) hallados.add([...ruta].sort().join('>'));
      else if (!ruta.includes(s) && ruta.length < maxLargo) dfs(ini, s, [...ruta, s]);
    }
  };
};
```

```
for (const n of sig.keys()) dfs(n, n, [n]);
return [...hallados].map((k) => k.split('>'));
}
```

Paso 3 · Identidad ERC-8004 del agente

- **Registro de identidad:** en Monad mainnet, 0x8004A169FB4a3325136EB29fA0ceB6D2e539a432. Cada agente es un NFT (ERC-721) cuyo tokenURI apunta a una "agent card".
- **Registro de reputación:** 0x8004BAa17C55a88189AE136b182e5fdA19dE9b63 (mainnet). Las direcciones de **testnet** no figuran en la guía de Monad: confirmarlas en github.com/erc-8004 o en el Discord antes de desplegar.
- **Agent card:** un JSON público (IPFS o una URL del repo) con nombre, descripción, endpoints y la wallet del agente.
- Registrar el agente con el SDK `agent0` o llamando directamente al contrato con `viem`; guardar el `agentId` resultante.

```
{
  "name": "Garante Auditor",
  "description": "Detecta anillos de calificaciones en Garante Protocol y publica veredictos firmados.",
  "endpoints": [{ "name": "veredictos", "endpoint": "https://<dominio>/api/veredictos" }],
  "wallet": "0x<direccion del agente>"
}
```

Paso 4 · Publicar el veredicto

El agente firma con `AGENT_PRIVATE_KEY` y llama a `GaranteAgentLog.publicar(agentId, cuentas, hashExplicacion)`. La explicación completa queda offchain y solo su hash va onchain. Envío indexa el evento y el perfil lo muestra.

Paso 5 · Otras funciones de Qwen

- Resumen del perfil para `(prestis)/reputacion/page.jsx`.
- Moderación del tablón (Plus): tasas abusivas, pedidos de depósito adelantado y enlaces sospechosos.

Tareas

Tarea	Quién	Estado	Dónde
Cuenta de Model Studio, créditos y API key	Alan	Pendiente	Alibaba Cloud
Cliente de Qwen y prompts de explicación y resumen	Alan	Pendiente	<code>agent/qwen.js</code>
Detector de anillos sobre el grafo de Envío	Alan	Pendiente	<code>agent/anillos.js</code>
Agent card y registro en el Identity Registry de ERC-8004	Alan	Pendiente	<code>agent/registro.js</code>
GaranteAgentLog (veredictos firmados)	Alan	Pendiente	<code>contracts/</code>
Mostrar alertas y resumen de IA en el perfil	Brenda	Pendiente	<code>(prestis)/reputacion/page.jsx</code>
Moderación del tablón con Qwen	Alan	Plus	<code>agent/moderar.js</code>

7. Bounty 5 · Best Workflow with CRE

Campo	Detalle
Patrocinador y premio	Chainlink · \$3,000
Qué es	Un workflow de Chainlink Runtime Environment (CRE) como capa de orquestación. Puede ser simulado.
Para qué lo hacemos	Un cron revisa las fechas límite de los préstamos activos, marca "Vencido" onchain y avisa al agente de Qwen. Es la parte que corre sola, sin que nadie haga clic.
Resultado esperado	Un workflow que, al correr (en simulación o en red), lee los préstamos activos, detecta los vencidos, llama a <code>marcarVencido(id)</code> y dispara al agente. Un registro de la ejecución como evidencia.
Riesgo	No sabemos si CRE funciona en Monad testnet. Por confirmar en el Discord. Si no, se entrega en simulación con el registro de la corrida.

Cómo empezar

1. Instalar **Bun 1.2.21 o más** (`bun --version`) y el **CRE CLI** (docs.chain.link/cre/getting-started/cli-installation).
2. Crear la cuenta de CRE y entrar: `cre login`. Comprobar con `cre whoami`.
3. Preguntar en el Discord de Monad si CRE soporta Monad testnet como red de lectura y escritura.

Procedimiento de implementación

Paso 1 · Crear el proyecto

```
cd cre
cre init
```

En el asistente: nombre del proyecto `garante-cre`, lenguaje **TypeScript**, plantilla **HelloWorld**, nombre del workflow `vencimientos` y registro `private`. Genera `project.yaml` (URLs de RPC), `secrets.yaml`, `.env` y la carpeta `vencimientos/` con `main.ts`, `workflow.yaml` y `config.staging.json`.

Paso 2 · Configurar

- `.env`: `CRE_ETH_PRIVATE_KEY=<64 caracteres hex, sin 0x>`. Nunca al repo.
- `config.staging.json`: el horario del cron, por ejemplo `{ "schedule": "0 */10 * * * *" }` (cada 10 minutos para la demo).
- `project.yaml`: agregar el RPC de Monad testnet si la red está soportada.

```
cd vencimientos && bun install
cd .. && cre workflow simulate vencimientos --target staging-settings
```

Paso 3 · Programar el workflow

1. Disparador: cron.
2. Leer de `GaranteLedger` los préstamos activos cuya fecha límite ya pasó.
3. Para cada uno, escribir `marcarVencido(id)` (en simulación usa `--broadcast` para enviar la transacción real; sin esa bandera solo simula).
4. Llamar por HTTP al agente de Qwen (`/api/vigilar`) para que revise el grafo y publique si hay algo.

El SDK de CRE tiene su propia API de triggers y de lectura/escritura onchain: seguir el ejemplo de la plantilla y la documentación (docs.chain.link/cre) en lugar de escribir el `main.ts` de memoria.

Tareas

Tarea	Quién	Estado	Dónde
Preguntar en el Discord si CRE soporta Monad testnet	Brenda	Pendiente	Discord
Instalar Bun y el CRE CLI, <code>cre login</code>	Alan	Pendiente	tu máquina
<code>cre init</code> y simulación del Helloworld	Alan	Pendiente	<code>cre/</code>
Workflow de vencimientos (lee, marca, avisa)	Alan	Pendiente	<code>cre/vencimientos/main.ts</code>
<code>marcarVencido</code> en el contrato	Alan	Pendiente	GaranteLedger
Guardar el registro de la corrida para la entrega	Alan	Pendiente	<code>docs/</code>

8. Bounty 6 · Best Projects Using Alchemy

Campo	Detalle
Patrocinador y premio	Alchemy · \$1,000 en créditos
Qué es	Integrar Alchemy de forma significativa.
Para qué lo hacemos	Usarlo como RPC de toda la app y del SDK, y recibir webhooks para los avisos de los préstamos. El RPC público de Monad es lento y limitado; el de Alchemy da más estabilidad para la demo.
Resultado esperado	Toda la app y el SDK leyendo por Alchemy, y un webhook que dispare un aviso cuando se abre o cierra un préstamo.
Prioridad	El RPC es lo primero y se puede hacer hoy. Los webhooks son lo primero que se suelta si hay atraso.

Cómo empezar

1. Crear cuenta en dashboard.alchemy.com y una app con la red **Monad Testnet**.
2. Copiar la API key. El endpoint tiene la forma `https://monad-testnet.g.alchemy.com/v2/<API_KEY>` (WebSocket: `wss://monad-testnet.g.alchemy.com/v2/<API_KEY>`).
3. Pedir los créditos de la bounty que ofrece el hackathon.

Procedimiento de implementación

Paso 1 · RPC (hoy)

Guardar la URL en `NEXT_PUBLIC_RPC_URL` y, en `app/src/lib/mera.js`, usarla en `monadTestnet` y en cada `http()`:

```
export const monadTestnet = defineChain({
  id: 10143, name: 'Monad Testnet',
  nativeCurrency: { name: 'Monad', symbol: 'MON', decimals: 18 },
  rpcUrls: { default: { http: [process.env.NEXT_PUBLIC_RPC_URL ?? 'https://testnet-rpc.monad.xyz'] } },
});
```

Paso 2 · Webhooks

1. En el dashboard: Webhooks → crear uno de tipo **Custom** sobre Monad Testnet filtrando los eventos de `GaranteLedger`, o **Address Activity** si basta con seguir direcciones.
2. Apuntarlo a una URL pública de la app: `https://<dominio>/api/avisos`. En local se necesita un túnel (por ejemplo ngrok).
3. Copiar la clave de firma a `ALCHEMY_WEBHOOK_SIGNING_KEY` y **verificar la firma** en la ruta antes de aceptar el aviso (el encabezado y el método exacto están en la documentación de webhooks de Alchemy; confirmarlo ahí).
4. La ruta `src/app/api/avisos/route.js` guarda o reenvía el aviso a la contraparte ("te abrieron un préstamo", "te lo cerraron").

Ojo con los avisos de "3 días y 1 día antes". Un webhook de Alchemy reacciona a **actividad onchain**; no sabe de calendario. Los avisos por fecha tienen que salir del cron de CRE (bounty 5), que llama a `/api/avisos`. Alchemy cubre los avisos por evento (préstamo abierto o cerrado).

Tareas

Tarea	Quién	Estado	Dónde
Cuenta, app en Monad Testnet y API key	Brenda	Pendiente	dashboard.alchemy.com
RPC de Alchemy en la app y en el SDK	Brenda	Pendiente	<code>src/lib/mera.js</code> , <code>.env.local</code>

Tarea	Quién	Estado	Dónde
Webhook de eventos del GaranteLedger	Brenda	Pendiente	dashboard de Alchemy
Ruta <code>/api/avisos</code> con verificación de firma	Brenda	Pendiente	<code>src/app/api/avisos/route.js</code>
Avisos 3 días y 1 día (disparados por CRE)	Los dos	Pendiente	CRE + <code>/api/avisos</code>

9. Bounty 7 · Best Community Team Project (por confirmar)

Campo	Detalle
Patrocinador y premio	Monad Foundation · \$5,000
Qué es	Premio para un equipo simpatizante de la comunidad de Metropolis.
Resultado esperado	Saber si contamos y, si sí, agregarla al proyecto.
Cómo empezar	Entrar al Discord de Monad Developers, conseguir el rol de Metropolis y preguntar en el canal del hackathon qué requisitos pide el premio y si podemos postular.

Tareas

Tarea	Quién	Estado	Dónde
Entrar al Discord y conseguir el rol de Metropolis	Brenda	Pendiente	Discord
Preguntar si contamos para Community Team y sus requisitos	Brenda	Pendiente	Discord
Agregarla en el panel si corresponde	Alan	Por confirmar	hackathon.monad.xyz

10. Construcción del Track 4: Trust, Identity & AI Infrastructure

Campo	Detalle
Qué es	La pista principal del hackathon. Premia el trabajo que es un protocolo o capa de infraestructura sobre la que otras apps se construyen, no un producto aislado. \$30,000 en tres ganadores de \$10,000 y la opción del Grand Champion (\$25,000).
Para qué lo hacemos	Todas las bounties anteriores son piezas del mismo protocolo. El Track 4 es donde se junta todo y se juzga como una sola cosa.
Resultado esperado	Un protocolo desplegado en Monad testnet con contratos verificados, un SDK que otra app ya usa, una app de ejemplo (Garante), una demo en vivo y la evidencia de mercado.
Pesos de los jueces	Ejecución técnica 20% · Diseño = experiencia del dev 20% · Originalidad 15% · Mercado y fundadores 25% · Tracción y camino a seguir 20%.

Cómo se construye, en orden

- Contratos** (Alan): GaranteIdentity, GaranteLedger, GaranteReputation, GaranteAgentLog. Pruebas en Foundry (código reusado, nonce repetido, doble calificación, carnet duplicado). Despliegue en Monad testnet y verificación en el explorer.
- Cuenta y firma** (Brenda): bounties 1 y 2. Entrar con huella, firmar el EIP-712 de Loan y mandar la transacción al contrato.
- Indexer** (Alan): bounty 3. Puntaje, nivel, perfiles, interesados.
- SDK `@garante/sdk`** (Alan): isVerified, getReputation, requestAccess, leyendo del indexer por RPC de Alchemy. Ejemplo de 10 líneas en sdk/examples/ y publicación en npm o como paquete del repo.
- App de ejemplo** (Brenda): conectar las pantallas del tablón, interesados, código, reputación y Mis Tratos a contratos, indexer y SDK. Retirar los datos fijos.
- Agente y workflow** (Alan): bounties 4 y 5. Anillos, veredictos ERC-8004, cron de vencimientos.
- Experiencia del dev**: playground (pegar una dirección y ver la reputación en vivo), badge "Verificado por Garante" y README que arranca con el ejemplo de 10 líneas.
- Dos integraciones del SDK**: una mini app propia (un "verificador de pasanaku" de unas 10 líneas) y otro equipo del hackathon. Pedirlo en el Discord desde ya.
- Evidencia de mercado**: 3 a 5 charlas con adoptantes de Cochabamba (cooperativa, organizador de pasanakus, prestamista informal) con una cita o carta de interés de cada uno, y una cifra verificable de ASFI o del Findex del Banco Mundial. No usar cifras estimadas.
- Modo juez y pruebas**: cuenta propia con huella, datos de muestra marcados, prueba stateless repetida en el despliegue final.
- Despliegue**: Vercel con el dominio final apuntando a testnet; repo público con acceso para metropolis@hackathon.monad.xyz.

Checklist de entrega

Tarea	Quién	Estado	Dónde
Logo (PNG, máx. 3 MB)	Brenda	Hecho	app/public/img/logo-condor.png (confirmar versión final)
Repo público, sin secretos ni datos personales	Alan	Pendiente	GitHub
Demo técnica, máx. 3 min, producto en vivo	Los dos	Pendiente	12 de oct
Video pitch, máx. 2 min	Los dos	Pendiente	12 de oct
Texto de Mera y video opcional de 2 min	Los dos	Pendiente	12 de oct

Tarea	Quién	Estado	Dónde
Link al producto + instrucciones + credenciales para jueces	Los dos	Pendiente	13 de oct
Perfil del proyecto: Track 4 y las bounties marcadas	Los dos	Pendiente	hackathon.monad.xyz
Enviar antes de las 18:00 (cierre real 23:59 GMT-4)	Los dos	Pendiente	13 de oct

11. Calendario

El producto se congela el **11 de octubre**; el 12 se graban los videos y el 13 se envía. Cada fase termina con un control: si no pasa, se recorta de los extras y no de la fase siguiente.

Fechas	Foco	Bounties	Control de salida
6 a 7 oct	Contratos desplegados, fondos de arranque, sesiones, indexer, SDK y pantallas con datos reales	1, 2, 3, 6 (RPC)	El SDK devuelve un perfil desde un script aparte; el tablón muestra interesados con referencias
8 a 9 oct	Agente de Qwen, cron de CRE, webhooks y segunda integración del SDK	4, 5, 6	Con 3 cuentas en círculo el agente publica la alerta y aparece en el perfil
10 a 11 oct	Modo juez, disputas, README, pulido y celular; congelar el producto	1, 2	Un tercero sigue el modo juez sin ayuda; primera toma del guion con cronómetro
12 oct	Grabar demo, pitch y video de Mera; subir los videos	Todas	Videos subidos
13 oct	Perfil del proyecto, instrucciones para jueces y envío	Todas	Envío antes de las 18:00

Reglas de trabajo

- Hacer pull antes de empezar y commits pequeños y diarios de los dos: los jueces revisan que el trabajo se hizo durante el hackathon.
- Ningún secreto en el repo: claves en `.env.local` y `.env`, y esos archivos en `.gitignore`. Ninguna clave en el navegador.
- Primero el MVP, después el Plus. Si una fase se atrasa, se recorta del Plus, nunca del modo juez ni de los videos.
- El chat del tablón es Plus: no se empieza hasta pasar el control del 7 de octubre.
- Si hay que soltar bounties, el orden es: webhooks de Alchemy primero, CRE queda simulado; las de Mera y Qwen no se tocan.
- Repetir la prueba stateless después de cada despliegue.
- Confirmar el nombre final antes de grabar los videos.
- Cada noche, 15 minutos de sincronización: qué se hizo, qué sigue, qué bloquea. Marcar cada tarea apenas se termine.
- El reparto Alan = contratos, datos y agente / Brenda = app y experiencia es una propuesta; cámbienlo según lo que cada uno domine.